Estructuras de Datos y Algoritmos

Algoritmos de ordenamiento

Dr. Jorge Armando Ramírez Díaz

Universidad Nacional Autónoma de México
ENES Unidad Juriquilla
Licenciatura en Matemáticas para el Desarrollo

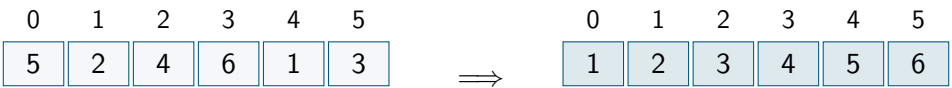
Ordenar

Problema de ordenamiento

Dado un arreglo A de n elementos comparables, reorganizarlos para obtener

$$A[0] \leq A[1] \leq \cdots \leq A[n-1].$$

El resultado debe conservar todos los elementos originales y sus repeticiones.



Los procedimientos modificarán el arreglo recibido. El vacío y el arreglo de un elemento ya están ordenados.

Estabilidad

Algoritmo estable

Un algoritmo de ordenamiento es estable si conserva el orden relativo de los elementos con la misma clave.

Entrada:

$$[2_a, 1, 2_b].$$

Conserva el orden entre iguales

$$[1, 2_a, 2_b]$$

2_a sigue antes que 2_b.

Invierte el orden entre iguales

$$[1, 2_b, 2_a]$$

Está ordenada, pero no respeta la estabilidad.

Un algoritmo estable garantiza esta conservación para cualquier entrada.

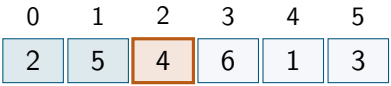
Inserción

El prefijo `A[0:i]` ya está ordenado.

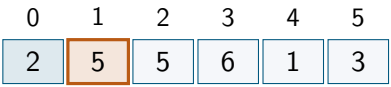
Tomamos `A[i]` como *clave* y desplazamos a la derecha los valores mayores.

La clave ocupa el espacio que queda libre.

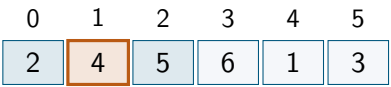
Antes de insertar 4 ($i = 2$)



5 se desplaza una posición



Insertamos 4 entre 2 y 5



La clave se guarda antes de modificar el arreglo.

Algoritmo de inserción

Procedimiento Insercion(A)

$n \leftarrow |A|$
Para i desde 1 hasta $n-1$:
  $clave \leftarrow A[i]$
  $j \leftarrow i-1$
  Mientras $j \geq 0$ y $A[j] > clave$:
    $A[j+1] \leftarrow A[j]$
    $j \leftarrow j-1$
  $A[j+1] \leftarrow clave$

```
def insercion(A):
    n = len(A)
    for i in range(1, n):
        clave = A[i]
        j = i - 1
        while j >= 0 and A[j] > clave:
            A[j+1] = A[j]
            j -= 1
        A[j+1] = clave
```

La función modifica A y devuelve `None`.

Costo: mejor caso $\Theta(n)$ con entrada ordenada; peor caso $\Theta(n^2)$ con entrada invertida.

Seguimiento

Inserción

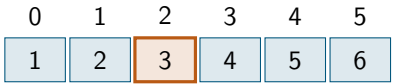
Entrada: $[5, 2, 4, 6, 1, 3]$.

En cada fila, el prefijo hasta i ya está ordenado.

En la iteración $i = 4$ del ciclo exterior, la clave 1 desplaza cuatro valores.

i	clave	**Después de la iteración**
1	2	$[\mathbf{2}, \mathbf{5}, 4, 6, 1, 3]$
2	4	$[\mathbf{2}, \mathbf{4}, \mathbf{5}, 6, 1, 3]$
3	6	$[\mathbf{2}, \mathbf{4}, \mathbf{5}, \mathbf{6}, 1, 3]$
4	1	$[\mathbf{1}, \mathbf{2}, \mathbf{4}, \mathbf{5}, \mathbf{6}, 3]$
5	3	$[\mathbf{1}, \mathbf{2}, \mathbf{3}, \mathbf{4}, \mathbf{5}, \mathbf{6}]$

Última inserción:



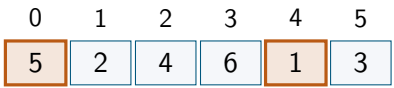
Selección

Buscamos el mínimo del sufijo `A[i:n]` y lo intercambiamos con `A[i]`.

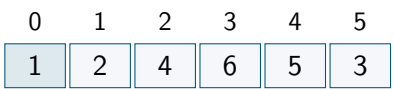
Después de cada iteración del ciclo exterior, una posición más queda en su lugar definitivo.

El prefijo contiene los menores elementos, en orden.

Primera búsqueda: mínimo 1 en $m = 4$



Intercambiamos las posiciones 0 y 4



La búsqueda siguiente comienza en $i = 1$.

Algoritmo de selección

Procedimiento Seleccion(A)

$n \leftarrow |A|$
Para i desde 0 hasta $n-2$:
  $m \leftarrow i$
  Para j desde $i+1$ hasta $n-1$:
    Si $A[j] < A[m]$:
      $m \leftarrow j$
  Intercambiar $A[i]$ con $A[m]$

```
def seleccion(A):
    n = len(A)
    for i in range(n - 1):
        m = i
        for j in range(i + 1, n):
            if A[j] < A[m]:
                m = j
        A[i], A[m] = A[m], A[i]
```

m guarda la posición del mínimo encontrado.

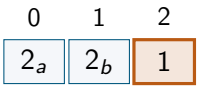
Costo: $\Theta(n^2)$ en el mejor y en el peor caso, independientemente del orden inicial.

Seguimiento

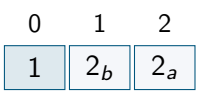
Selección

i	m	**Después de la iteración**
0	4	$[\mathbf{1}, 2, 4, 6, 5, 3]$
1	1	$[\mathbf{1}, \mathbf{2}, 4, 6, 5, 3]$
2	5	$[\mathbf{1}, \mathbf{2}, \mathbf{3}, 6, 5, 4]$
3	5	$[\mathbf{1}, \mathbf{2}, \mathbf{3}, \mathbf{4}, 5, 6]$
4	4	$[\mathbf{1}, \mathbf{2}, \mathbf{3}, \mathbf{4}, \mathbf{5}, 6]$

Selección no estable



Intercambiar el mínimo con $A[0]$



2_a estaba antes de 2_b; ahora está después.

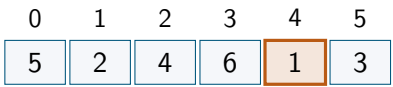
Burbuja

Comparamos vecinos de derecha a izquierda e intercambiamos si están invertidos.

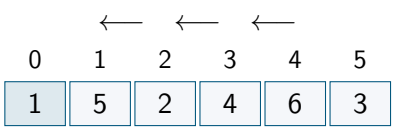
El mínimo del sufijo avanza hasta la posición i.

Al terminar una iteración del ciclo exterior, el prefijo ordenado crece una posición.

Primera iteración exterior: $j = 5, 4, 3, 2, 1$



El 1 cruza a sus vecinos mayores



La siguiente iteración exterior deja fijo el mínimo de `A[1:n]`.

Algoritmo de burbuja

Procedimiento Burbuja(A)

$n \leftarrow |A|$
Para i desde 0 hasta $n-2$:
  Para j desde $n-1$ hasta $i+1$,
  en orden decreciente:
    Si $A[j] < A[j-1]$:
      Intercambiar $A[j]$ con $A[j-1]$

```
def burbuja(A):
    n = len(A)
    for i in range(n - 1):
        for j in range(n - 1, i, -1):
            if A[j] < A[j-1]:
                A[j], A[j-1] = (
                    A[j-1], A[j]
                )
```

El prefijo anterior a i contiene los menores elementos, ya ordenados.

Costo: $\Theta(n^2)$ en el mejor y en el peor caso: se completan todos los recorridos, incluso si no hay intercambios.

Burbuja

Entrada: $[5, 2, 4, 6, 1, 3]$.

Cada iteración del ciclo exterior fija el mínimo del sufijo pendiente.

Después de $i = 3$, el arreglo ya está ordenado. La iteración $i = 4$ se ejecuta de todos modos.

i	**Después de la iteración**
0	$[\mathbf{1}, 5, 2, 4, 6, 3]$
1	$[\mathbf{1}, \mathbf{2}, 5, 3, 4, 6]$
2	$[\mathbf{1}, \mathbf{2}, \mathbf{3}, 5, 4, 6]$
3	$[\mathbf{1}, \mathbf{2}, \mathbf{3}, \mathbf{4}, 5, 6]$
4	$[\mathbf{1}, \mathbf{2}, \mathbf{3}, \mathbf{4}, \mathbf{5}, 6]$

Shellsort

Ordenamos por inserción las subsecuencias separadas por una distancia h. Un arreglo está *h-ordenado* si $A[j-h] \leq A[j]$ para todo $j \geq h$.

Con $h = 4$, las posiciones $0, 4, 8$ forman una subsecuencia:

0	1	2	3	4	5	6	7	8
9	8	7	6	5	4	3	2	1

Al terminar la iteración con $h = 4$:

0	1	2	3	4	5	6	7	8
1	4	3	2	5	8	7	6	9

Reducimos h hasta 1. Con $h = 1$, ordenar la subsecuencia significa ordenar todo el arreglo.

Algoritmo de Shellsort

Procedimiento Shellsort(A)

$n \leftarrow |A|$, $h \leftarrow 1$
Mientras $h < \lfloor n/3 \rfloor$:
  $h \leftarrow 3h + 1$
Mientras $h \geq 1$:
  Para i desde h hasta $n - 1$:
    $j \leftarrow i$
    Mientras $j \geq h$ y $A[j] < A[j - h]$:
      Intercambiar $A[j]$ con $A[j - h]$
      $j \leftarrow j - h$
  $h \leftarrow \lfloor h/3 \rfloor$

Saltos $3h + 1$

Se generan en orden creciente: $1, 4, 13, 40, \ldots$

Se utilizan en orden decreciente. Para $n = 9$, son $4, 1$.

Cada iteración con un salto h ordena por inserción las subsecuencias separadas esa distancia. La última iteración usa $h = 1$.

Shellsort en Python

```
def shellsort(A):
    n = len(A)
    h = 1
    while h < n // 3:
        h = 3*h + 1
    while h >= 1:
        for i in range(h, n):
            j = i
            while (j >= h
                   and A[j] < A[j-h]):
                A[j], A[j-h] = (
                    A[j-h], A[j])
                j -= h
        h //= 3
```

`j -= h` recorre hacia atrás una subsecuencia.

La guarda evita índices negativos antes de consultar `A[j-h]`.

`h //= 3` recupera el salto anterior de la sucesión.

Solo usa variables auxiliares: modifica `A` y devuelve `None`.

Sucesiones y costos

Sucesión	**Primeros términos**	**Mejor caso**	**Peor caso**
Shell original	$\lfloor n/2 \rfloor, \lfloor n/4 \rfloor, \ldots, 1$	$\Theta(n \log_2 n)$	$O(n^2)$
Hibbard	$1, 3, 7, 15, 31, \ldots$	$\Theta(n \log_2 n)$	$\Theta(n^{3/2})$
Knuth	$1, 4, 13, 40, 121, \ldots$	$\Theta(n \log_2 n)$	$\Theta(n^{3/2})$
Pratt	$1, 2, 3, 4, 6, 8, 9, 12, \ldots$	$\Theta(n(\log_2 n)^2)$	$\Theta(n(\log_2 n)^2)$
Sedgewick	$1, 8, 23, 77, 281, \ldots$	$\Theta(n \log_2 n)$	$O(n^{4/3})$

Seguimiento

Shellsort con $n = 9$

Saltos: 4 y 1.

Para $h = 4$, las subsecuencias usan las posiciones

$$(0, 4, 8),\ (1, 5),\ (2, 6),\ (3, 7).$$

La primera pasa de $(9, 5, 1)$ a $(1, 5, 9)$.

Las iteraciones con $h = 4$ y $h = 1$ hacen 6 intercambios cada una en esta entrada.

Estado	**Arreglo**
Inicial	$[9, 8, 7, 6, 5, 4, 3, 2, 1]$
Fin $h = 4$	$[1, 4, 3, 2, 5, 8, 7, 6, 9]$
$h = 1$, $i = 3$	$[1, 2, 3, 4, 5, 8, 7, 6, 9]$
Fin $h = 1$	$[1, 2, 3, 4, 5, 6, 7, 8, 9]$

Después de $h = 4$, el arreglo aún no está ordenado por completo.

Heapsort

El máximo del heap está en la raíz. Lo intercambiamos con el último elemento activo y reservamos esa posición para el resultado.

Max-heap inicial

0	1	2	3	4	5	6	7	8	9
16	14	10	8	7	9	3	2	4	1

Después del intercambio, reducir el tamaño y hundir

0	1	2	3	4	5	6	7	8	9
14	8	10	4	7	9	3	2	1	16

$\underbrace{A[0 \ldots 8]}_{\text{heap activo}}$ $\underbrace{A[9]}_{\text{sufijo ordenado}}$.

Los elementos del sufijo permanecen en el arreglo y dejan de pertenecer al heap activo.

Algoritmo de heapsort

Procedimiento Heapsort(A)

ConstruirHeap(A)
$n \leftarrow |A|$
Para i desde $n-1$ hasta 1, decreciendo:
  Intercambiar $A[0]$ con $A[i]$
  $n \leftarrow n-1$
  Hundir($A, n, 0$)

```
def heapsort(A):
    construir_heap(A)
    n = len(A)
    for i in range(n - 1, 0, -1):
        A[0], A[i] = A[i], A[0]
        n -= 1
        hundir(A, n, 0)
```

El prefijo se reduce, mientras el sufijo ordenado crece. La longitud de A permanece igual y el procedimiento devuelve `None`.

Costo de heapsort

Construir: $\Theta(n)$; `hundir` con tamaño activo m: $O(\log_2(m+1))$. Sea $n = |A| \geq 2$ el tamaño inicial; $a, b > 0$ son constantes.

Hay $n-1$ términos, cada uno menor o igual que $\log_2(n+1)$.

$$\begin{aligned}&\sum_{m=1}^{n-1} \log_2(m+1)\\ &= \log_2 2 + \cdots + \log_2 n\\ &\leq (n-1)\log_2(n+1).\end{aligned}$$

$n - 1 \leq n$ y $1 \leq \log_2(n+1)$.

$$\begin{aligned}T(n) &\leq an + b\sum_{m=1}^{n-1} \log_2(m+1)\\ &\leq an + b(n-1)\log_2(n+1)\end{aligned}$$

Cota superior: $O(n\log_2(n+1))$. **Peor caso:** $\Theta(n\log_2(n+1))$.

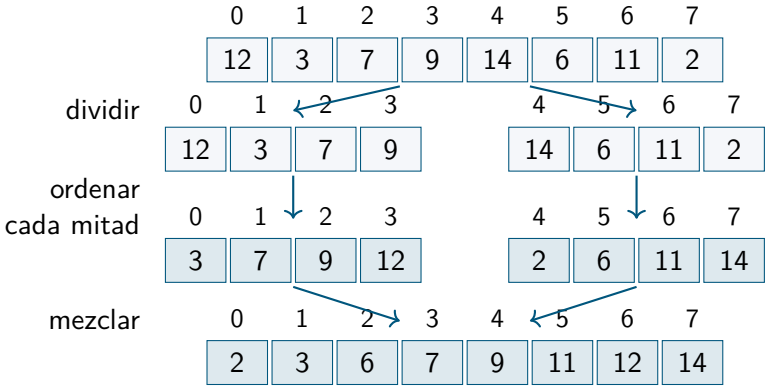
Seguimiento

Paso	Heap activo	Sufijo ordenado
Inicial	$16, 14, 10, 8, 7, 9, 3, 2, 4, 1$	vacío
$i = 9$	$14, 8, 10, 4, 7, 9, 3, 2, 1$	16
$i = 8$	$10, 8, 9, 4, 7, 1, 3, 2$	$14, 16$
$i = 7$	$9, 8, 3, 4, 7, 1, 2$	$10, 14, 16$
$\vdots$	$\vdots$	$\vdots$
$i = 1$	1	$2, 3, 4, 7, 8, 9, 10, 14, 16$

Resultado: $A = [1, 2, 3, 4, 7, 8, 9, 10, 14, 16]$.

Ordenamiento por mezcla

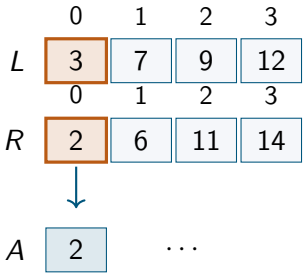
Dividimos el intervalo, ordenamos cada mitad y mezclamos sus resultados.



Mezclar

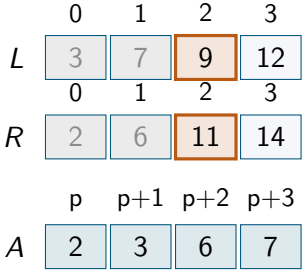
Solo comparamos los primeros valores que quedan en L y en R.

Primer valor: $\text{mín}(3, 2) = 2$



Copiamos 2 a $A[p]$. Avanzan j y k.

Después de cuatro escrituras



Ahora $i = j = 2$, $k = p + 4$. El siguiente valor es $\text{mín}(9, 11) = 9$.

Algoritmo de mezcla

$A[p..q]$ y $A[q+1..r]$ están ordenados, con $0 \leq p \leq q < r < |A|$.

Procedimiento Mezclar(A, p, q, r): primera parte

$L \leftarrow$ copia de $A[p..q]$
$R \leftarrow$ copia de $A[q+1..r]$
$i \leftarrow 0$, $j \leftarrow 0$, $k \leftarrow p$
mientras $i < |L|$ **y** $j < |R|$
  si $L[i] \leq R[j]$
    $A[k] \leftarrow L[i]$
    $i \leftarrow i + 1$
  en otro caso
    $A[k] \leftarrow R[j]$
    $j \leftarrow j + 1$
  $k \leftarrow k + 1$

El ciclo termina cuando uno de los temporales se agota.

Mezcla en Python

Los cortes crean los temporales. El extremo derecho del corte se excluye.

```
def mezclar(A, p, q, r):
    L = A[p:q + 1]
    R = A[q + 1:r + 1]
    i = j = 0
    k = p
    while i < len(L) and j < len(R):
        if L[i] <= R[j]:
            A[k] = L[i]
            i += 1
        else:
            A[k] = R[j]
            j += 1
        k += 1
```

La función continúa en la siguiente diapositiva.

Copiar los restantes

Al terminar el ciclo principal, copiamos lo que queda del otro temporal.

Procedimiento Mezclar: continuación

mientras $i < |L|$
  $A[k] \leftarrow L[i]$
  $i \leftarrow i + 1$
  $k \leftarrow k + 1$

mientras $j < |R|$
  $A[k] \leftarrow R[j]$
  $j \leftarrow j + 1$
  $k \leftarrow k + 1$

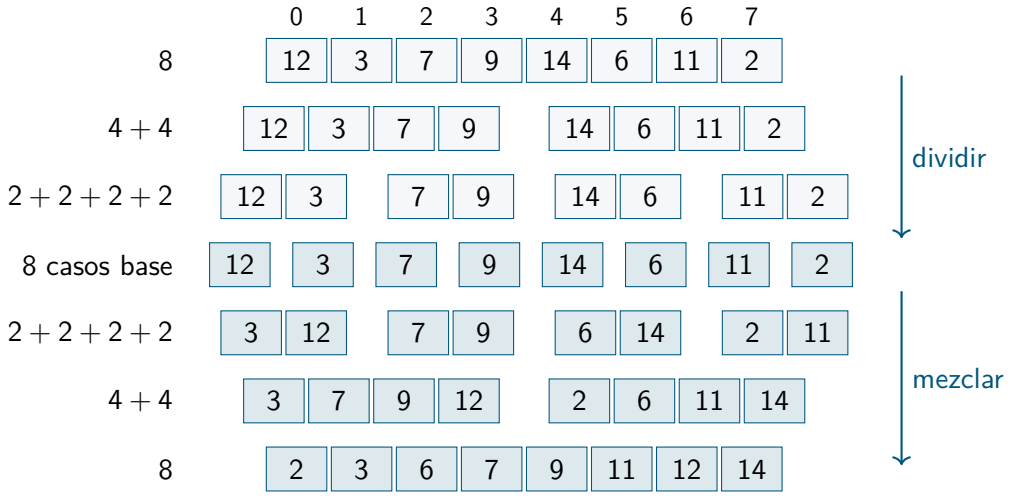
Python: continuación de `mezclar`

```
    while i < len(L):
        A[k] = L[i]
        i += 1
        k += 1

    while j < len(R):
        A[k] = R[j]
        j += 1
        k += 1
```

Sea m, el número de elementos que mezclamos. **Costo:** $\Theta(m)$.

Seguimiento



Algoritmo de mergesort

Ordenar $A[p..r]$, con ambos extremos inclusivos. Si $p \geq r$, el intervalo tiene cero o un elemento.

Procedimiento Mergesort(A, p, r)

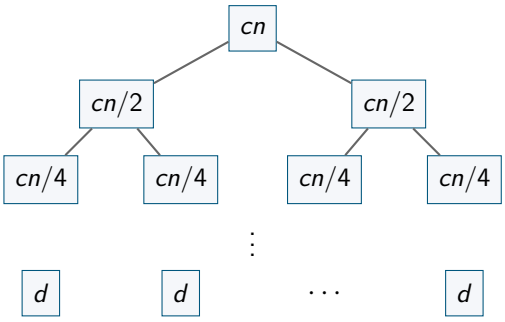
si $p \geq r$
  retornar
$q \leftarrow \lfloor (p + r)/2 \rfloor$
Mergesort(A, p, q)
Mergesort($A, q + 1, r$)
Mezclar(A, p, q, r)

```
def mergesort(A, p, r):
    if p >= r:
        return
    q = (p + r) // 2
    mergesort(A, p, q)
    mergesort(A, q + 1, r)
    mezclar(A, p, q, r)
```

Llamada inicial: `mergesort(A, 0, len(A) - 1)`. El resultado queda en A y la función retorna `None`.

Árbol de costos

$T(n) = 2T(n/2) + cn,\ T(1) = d$, con $n = 2^h$.



n hojas, costo conjunto dn

Nivel interno i, $0 \leq i < h$

Nodos	2^i
Tamaño por nodo	$n/2^i$
Costo por nodo	$cn/2^i$
Costo del nivel	cn

La división termina cuando

$$\frac{n}{2^h} = 1 \quad \Longrightarrow \quad h = \log_2 n.$$

Hay h niveles internos y un nivel de hojas.

Costo total

Para el modelo, con $n = 2^h$, sumamos niveles internos y hojas:

$$T(n) = \sum_{i=0}^{h-1} cn + dn = cnh + dn = cn \log_2 n + dn.$$

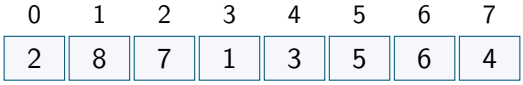
Para $n \geq 2$, se cumple $\log_2 n \geq 1$. Por tanto,

$$\underbrace{cn \log_2 n}_{\text{cota inferior}} \leq T(n) \leq \underbrace{(c+d) n \log_2 n}_{\text{cota superior}}.$$

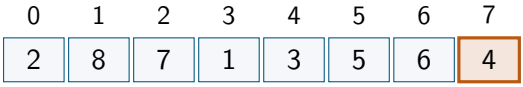
$$\boxed{T(n) \in \Theta(n \log_2 n)}$$

Quicksort

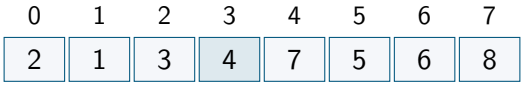
Arreglo original



Elegimos el último elemento como pivote: 4



Después de particionar



El pivote queda en su posición final: a su izquierda hay claves menores o iguales que 4; a su derecha, mayores que 4.

Quicksort ordena recursivamente ambas regiones, excluyendo el pivote.

Particionar

Trabajamos en $A[p \ldots r]$, con extremos inclusivos. El pivote es $x = A[r]$.

Antes de examinar la posición j

$$\underbrace{A[p \ldots i]}_{\leq x} \quad \underbrace{A[i+1 \ldots j-1]}_{> x} \quad \underbrace{A[j \ldots r-1]}_{\text{por examinar}} \quad \underbrace{A[r]}_{x}$$

- Si $A[j] \leq x$, extendemos la primera región y trasladamos allí esa clave.
- Si $A[j] > x$, la segunda región se extiende al avanzar j.
- Al terminar, intercambiamos el pivote con $A[i+1]$.

Al inicio, $i = p - 1$ y $j = p$: las dos regiones clasificadas están vacías. Un intervalo cuyo inicio supera al final representa una región vacía.

Algoritmo de partición

$0 \leq p \leq r < \text{len}(A)$. Devuelve el índice final del pivote.

Función Particionar(A, p, r)

$x \leftarrow A[r]$
$i \leftarrow p - 1$
Para j desde p hasta $r - 1$:
  Si $A[j] \leq x$:
    $i \leftarrow i + 1$
    Intercambiar $A[i]$ y $A[j]$.
Intercambiar $A[i + 1]$ y $A[r]$.
Retornar $i + 1$.

```
def particionar(A, p, r):
    x = A[r]
    i = p - 1
    for j in range(p, r):
        if A[j] <= x:
            i += 1
            A[i], A[j] = A[j], A[i]
    A[i + 1], A[r] = A[r], A[i + 1]
    return i + 1
```

Si devuelve q, entonces $A[p \ldots q - 1] \leq A[q]$ y $A[q + 1 \ldots r] > A[q]$. **Costo:** $\Theta(n)$, con $n = r - p + 1$ elementos en el subarreglo.

Seguimiento

Aplicamos `particionar(A, 0, 7)`; el pivote es $x = 4$.

Paso	i al terminar	Claves en el arreglo
Inicial	-1	2, 8, 7, 1, 3, 5, 6, 4
$j = 0$	0	2, 8, 7, 1, 3, 5, 6, 4
$j = 1$	0	2, 8, 7, 1, 3, 5, 6, 4
$j = 2$	0	2, 8, 7, 1, 3, 5, 6, 4
$j = 3$	1	2, 1, 7, 8, 3, 5, 6, 4
$j = 4$	2	2, 1, 3, 8, 7, 5, 6, 4
$j = 5$	2	2, 1, 3, 8, 7, 5, 6, 4
$j = 6$	2	2, 1, 3, 8, 7, 5, 6, 4
Intercambio final	2	2, 1, 3, **4**, 7, 5, 6, 8

La función devuelve $q = 3$. El 4 ocupa su posición final; las dos regiones todavía deben ordenarse.

Algoritmo de quicksort

Las llamadas recursivas excluyen al pivote que ya ocupa su posición final.

Procedimiento Quicksort(A, p, r)

Si $p \geq r$:
  Retornar.
$q \leftarrow$ Particionar(A, p, r)
Quicksort$(A, p, q-1)$
Quicksort$(A, q+1, r)$

```
def quicksort(A, p, r):
    if p >= r:
        return
    q = particionar(A, p, r)
    quicksort(A, p, q - 1)
    quicksort(A, q + 1, r)
```

Los segmentos de cero o un elemento ya están ordenados.

Llamada inicial: `quicksort(A, 0, len(A) - 1)`.

Modifica `A` y devuelve `None`. Para `A = []`, recibe $p = 0$, $r = -1$ y termina directamente.

Recurrencia

En una llamada sobre $A[p \dots r]$, definimos

$$n = r - p + 1, \qquad k = q - p.$$

Región	Número de elementos
Izquierda: $p, \dots, q-1$	k
Pivote: q	1
Derecha: $q+1, \dots, r$	$n-k-1$

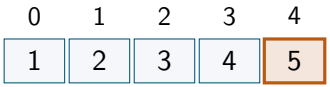
Modelo de costo para $n \geq 2$

$$T(n) = T(k) + T(n-k-1) + cn, \qquad 0 \leq k \leq n-1.$$

Con $c > 0$, cn modela el trabajo lineal local. **Casos base:** $T(0) = T(1) = d > 0$. q es un **índice**; k, una **cantidad** que depende de las claves y del pivote.

Peor caso

Tomemos una entrada estrictamente creciente y elijamos el último elemento:



El pivote 5 es el máximo: todos los demás elementos quedan a su izquierda. La partición devuelve $q = r$. Las regiones resultantes son:

$$\underbrace{[1,2,3,4]}_{\text{izquierda: 4 elementos}} \qquad \underbrace{5}_{\text{pivote}} \qquad \underbrace{[\,]}_{\text{derecha: 0 elementos}} .$$

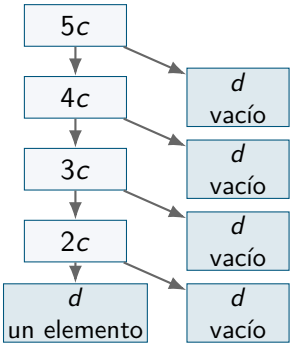
La siguiente llamada no vacía recibe $[1,2,3,4]$, que sigue creciente. Ahora el pivote 4 vuelve a ser el máximo, y el patrón se repite.

$$T(n) = T(n-1) + T(0) + cn, \qquad T(0) = T(1) = d.$$

En cada partición quedan $n-1$ elementos a la izquierda y ninguno a la derecha.

Árbol de costos

Ejemplo con 5 elementos. Las etiquetas son costos locales; cada hoja cuesta d.



Para $n \geq 2$ elementos

Hay $n-1$ nodos internos, con tamaños $n, n-1, \ldots, 2$ y costos $cn, c(n-1), \ldots, 2c$.

Hay $n-1$ hojas vacías y una hoja con un elemento. Cada hoja cuesta d: $(n-1)d + d = nd$.

Costo total

$$T(n) = c \sum_{m=2}^{n} m + nd.$$

Costo del peor caso

Sumamos el trabajo de los nodos internos y el costo de todas las hojas:

$$2 + 3 + \cdots + n = (1 + 2 + \cdots + n) - 1 = \frac{n(n+1)}{2} - 1.$$

$$\begin{aligned} T(n) &= c \sum_{m=2}^{n} m + nd \\ &= c\left(\frac{n(n+1)}{2} - 1\right) + nd \\ &= \frac{c}{2} n(n+1) - c + nd \\ &= \frac{c}{2} n^2 + \frac{c}{2} n - c + nd \\ &= \frac{c}{2} n^2 + \left(\frac{c}{2} + d\right) n - c. \end{aligned}$$

Con $c > 0$, esta entrada cuesta $\Theta(n^2)$. La cota general $O(n^2)$ da $\boxed{T_{\text{peor}}(n) \in \Theta(n^2)}$.

Particiones equilibradas

Supongamos que **cada** partición deja regiones de tamaños tan iguales como sea posible:

$$T(n) = T\left(\left\lfloor \frac{n-1}{2} \right\rfloor\right) + T\left(\left\lceil \frac{n-1}{2} \right\rceil\right) + cn.$$

Trabajo por niveles

- Una llamada con m elementos aporta un costo local cm.
- En cada nivel, los tamaños suman a lo sumo n: el costo local conjunto es a lo sumo cn.
- Hay $\Theta(\log_2 n)$ niveles superiores con costo $\Theta(n)$ por nivel.

Sumando los niveles y el costo $O(n)$ de las llamadas base:

$$\boxed{T(n) \in \Theta(n \log_2 n).}$$

El pivote queda fuera de las llamadas; los redondeos conservan la cota asintótica.

Pivote aleatorio

En cada subproblema, elegimos uniformemente un índice entre p y r.

Función ParticionarAleatorio(A, p, r)

$s \leftarrow$ EnteroUniforme(p, r)
Intercambiar $A[s]$ y $A[r]$.
Retornar Particionar(A, p, r).

Procedimiento QuicksortAleatorio(A, p, r)

Si $p \geq r$:
  Retornar.
$q \leftarrow$ ParticionarAleatorio(A, p, r)
QuicksortAleatorio$(A, p, q-1)$
QuicksortAleatorio$(A, q+1, r)$

```python
from random import randint

def particionar_aleatorio(A, p, r):
    s = randint(p, r)
    A[s], A[r] = A[r], A[s]
    return particionar(A, p, r)

def quicksort_aleatorio(A, p, r):
    if p >= r:
        return
    q = particionar_aleatorio(A, p, r)
    quicksort_aleatorio(A, p, q - 1)
    quicksort_aleatorio(A, q + 1, r)
```

Costos y elección

Para las implementaciones estudiadas, con n elementos:

Algoritmo	Mejor caso	Peor caso	Estable	Cuándo conviene
Inserción	$\Theta(n)$	$\Theta(n^2)$	Sí	Entradas pequeñas; uso didáctico.
Selección	$\Theta(n^2)$	$\Theta(n^2)$	No	Entradas pequeñas; uso didáctico.
Burbuja	$\Theta(n^2)$	$\Theta(n^2)$	Sí	Entradas pequeñas; uso didáctico.
Shell, $3h+1$	$\Theta(n \log_2 n)$	$O(n^{3/2})$	No	Alternativa basada en inserción que mejora la cota de peor caso de los métodos cuadráticos.
Heap	$\Theta(n)$	$\Theta(n \log_2 n)$	No	Cuando se necesita garantizar $O(n \log_2 n)$ en el peor caso.
Merge	$\Theta(n \log_2 n)$	$\Theta(n \log_2 n)$	Sí	Cuando se requiere estabilidad y costo $O(n \log_2 n)$.
Quick	$\Theta(n \log_2 n)$	$\Theta(n^2)$	No	Para uso general, cuando se esperan particiones razonablemente equilibradas.
`list.sort()`	$\Theta(n)$	$\Theta(n \log_2 n)$	Sí	Uso general en Python.

CPython: Timsort; desde Python 3.11, Powersort decide el orden de las mezclas.